Java Collections Framework

Introduction

- A *collection* — sometimes called a container — is simply an object that groups multiple elements into a single unit.

- Collections are used to store, retrieve, manipulate, and communicate aggregate data.

Collections Framework

- A *collections framework* is a unified architecture for representing and manipulating collections. All collections frameworks contain the following:
  - **Interfaces:** These are abstract data types that represent collections. Interfaces allow collections to be manipulated independently of the details of their representation. In object-oriented languages, interfaces generally form a hierarchy.
  - **Implementations:** These are the concrete implementations of the collection interfaces. In essence, they are reusable data structures.
  - **Algorithms:** These are the methods that perform useful computations, such as searching and sorting, on objects that implement collection interfaces. The algorithms are said to be *polymorphic*: that is, the same method can be used on many different implementations of the appropriate collection interface. In essence, algorithms are reusable functionality.

Collections Framework

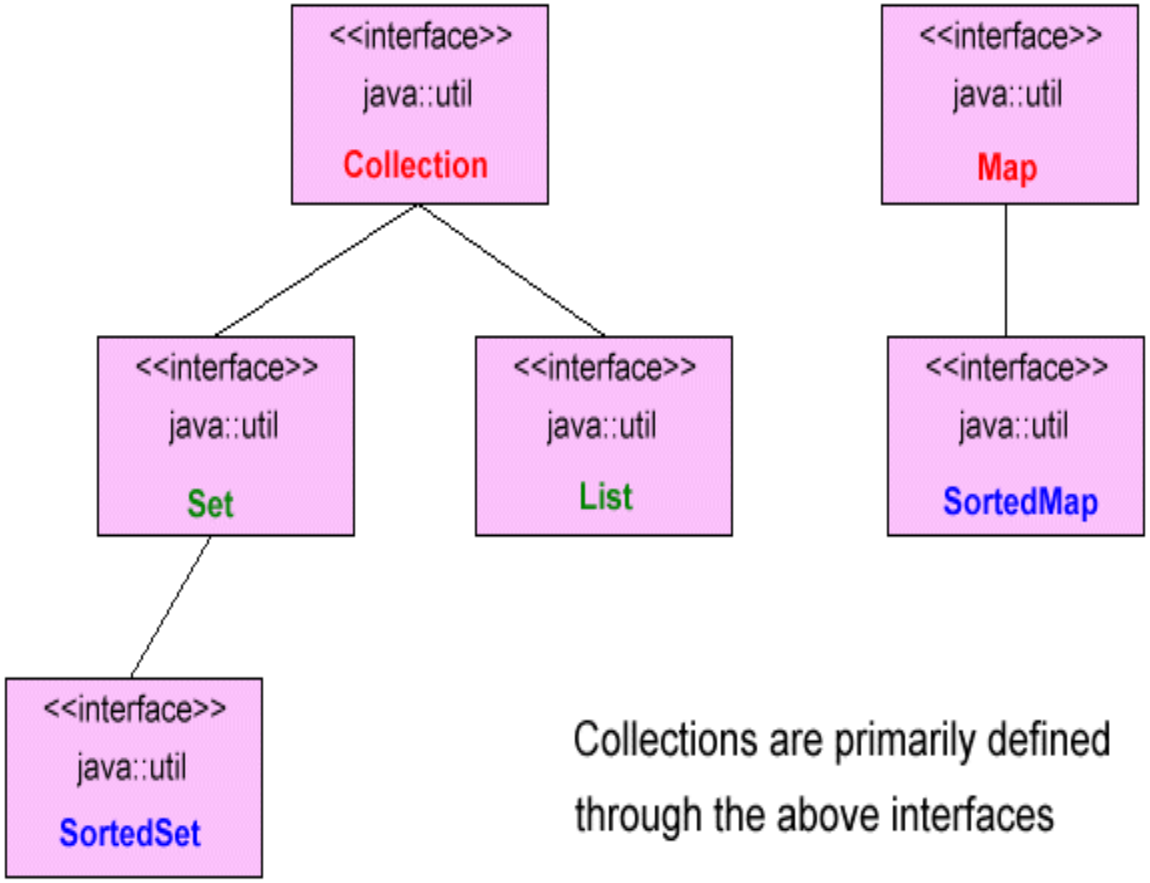
- Apart from the Java Collections Framework, the best-known examples of collections frameworks are:
  - C++ Standard Template Library (STL)
  - Smalltalk's collection hierarchy.

Benefits

- **Reduces effort to design new APIs**
- **Reduces effort to learn and to use new APIs**
- **Reduces programming effort**
- **Increases program speed and quality**
- **Fosters software reuse**

Overview: Core Interfaces

- Collection
- Set
- List
- Map
- SortedSet
- SortedMap



Collections are primarily defined through the above interfaces

Core Interface Hierarchy

- One might think that Map would extend Collection.
- In mathematics, a map is just a collection of pairs.
- In the Collections Framework, however, the interfaces Map and Collection are distinct with no lineage in the hierarchy.

Core Interface Hierarchy

- When designing software with the Collections Framework, it is useful to remember the following hierarchical relationships of the four basic interfaces of the framework:
  - The Collection interface is a group of objects, with duplicates allowed.
  - The Set interface extends Collection but forbids duplicates.
  - The List interface extends Collection, allows duplicates, and introduces positional indexing.
  - The Map interface extends neither Set nor Collection.

JCF implementations vs Historical Implementations

Interface	Implementation	Historical
Set	HashSet	
	TreeSet	
List	ArrayList	Vector
	LinkedList	Stack
Map	HashMap	Hashtable
	TreeMap	Properties

Collection

- A Collection represents a group of objects known as its elements.
- The Collection interface is used to pass around collections of objects where maximum generality is desired. For example, by convention all general-purpose collection implementations have a constructor that takes a Collection argument.
- This constructor, known as a *conversion constructor*, initializes the new collection to contain all of the elements in the specified collection, whatever the given collection's subinterface or implementation type.
- In other words, it allows you to convert the collection's type.

Collection

- Suppose, for example, that you have a Collection<String> c, which may be a List, a Set, or another kind of Collection.

- This idiom creates a new ArrayList (an implementation of the List interface), initially containing all the elements in c.

```
List<String> list = new ArrayList<String>(c);
```

Traversing Collection

- There are two ways to traverse collection:
  - with a for loop
  - by using Iterators

Traversing Collection with for loop

- The following code uses the for-each construct to print out each element of a collection on a separate line:

```
for (Object o : collection)
System.out.println(o);
```

iterators

- An Iterator is an object that enables you to traverse through a collection and to remove elements from the collection selectively, if desired.

- The following is the Iterator interface:

```
public interface Iterator<E>
{
 boolean hasNext();
 E next();
void remove(); //optional
}
```

Iterator Methods

- boolean hasNext()
  - Returns true if the iteration has more elements
- Object next()
  - Returns next element in the iteration
- void remove()
  - Removes the current element from the underlying Collection

Set

- interface Set extends Collection
- An unordered collection of objects
- No duplicate elements
- Same methods as Collection
  - Semantics are different, so different interface needed for design
- Implemented by:
  - HashSet, TreeSet

Set

- To demonstrate the use of the concrete Set classes, the following program creates a HashSet and adds a group of names, including one name twice.
- The program then prints out the list of names in the set, demonstrating the duplicate name isn't present.
- Next, the program treats the set as a TreeSet and displays the list sorted.

Set

```
import java.util.*;

 public class SetExample {
   public static void main(String args[]) {
     Set set = new HashSet();
     set.add("Bernadine");
     set.add("Elizabeth");
     set.add("Gene");
     set.add("Elizabeth");
     set.add("Clara");
     System.out.println(set);
     Set sortedSet = new TreeSet(set);
     System.out.println(sortedSet);
   }
 }
```

Running the program produces the following output. Notice that the duplicate entry is only present once, and the second list output is sorted alphabetically.

```
[Gene, Clara, Bernadine, Elizabeth]
 [Bernadine, Clara, Elizabeth, Gene]
```

List

- interface List extends Collection
- An ordered collection of objects
- Duplicates allowed

List Details

- Major additional methods:
  - **Object get(int);**
  - **Object set(int, Object);**
  - **int indexOf(Object);**
  - **int lastIndexOf(Object);**
  - **void add(int, Object);**
  - **Object remove(int);**
  - **List subList(int, int);**
- Implemented by:
  - ArrayList, LinkedList, Vector, Stack

List

- The following program demonstrates the use of the concrete List classes.
- The first part creates a List backed by an ArrayList. After filling the list, specific entries are retrieved.
- The LinkedList part of the example treats the LinkedList as a queue, adding things at the beginning of the queue and removing them from the end.

```
import java.util.*;

public class ListExample {
  public static void main(String args[]) {
    List list = new ArrayList();
    list.add("Bernadine");
    list.add("Elizabeth");
    list.add("Gene");
    list.add("Elizabeth");
    list.add("Clara");
    System.out.println(list);
    System.out.println("2: " + list.get(2));
    System.out.println("0: " + list.get(0));
    LinkedList queue = new LinkedList();
    queue.addFirst("Bernadine");
    queue.addFirst("Elizabeth");
    queue.addFirst("Gene");
    queue.addFirst("Elizabeth");
    queue.addFirst("Clara");
    System.out.println(queue);
    queue.removeLast();
    queue.removeLast();
    System.out.println(queue);
  }
}
```

Running the program produces the following output. Notice that unlike `Set`, `List` permits duplicates.

```
[Bernadine, Elizabeth, Gene, Elizabeth, Clara]
 2: Gene
 0: Bernadine
 [Clara, Elizabeth, Gene, Elizabeth, Bernadine]
 [Clara, Elizabeth, Gene]
```

Map

- interface Map (does <u>not</u> extend Collection)
- An object that maps keys to values
- Each key can have at most one value
- Replaces java.util.Dictionary interface
- Ordering may be provided by implementation class, but not guaranteed

Map Details

- Major methods:
  - **int size();**
  - **boolean isEmpty();**
  - **boolean containsKey(Object);**
  - **boolean containsValue(Object);**
  - **Object get(Object);**
  - **Object put(Object, Object);**
  - **Object remove(Object);**
  - **void putAll(Map);**
  - **void clear();**
- Implemented by:
  - HashMap, TreeMap, Hashtable, Properties

Accessing all members of Map

- Methods
  - `Set keySet();`
  - `Collection values();`
  - `Set entrySet();`

Map

- The following program demonstrates the use of the concrete Map classes.
- The program generates a frequency count of words passed from the command line.
- A HashMap is initially used for data storage. Afterwards, the map is converted to a TreeMap to display the key list sorted.

Map

```
import java.util.*;

 public class MapExample {
   public static void main(String args[]) {
     Map map = new HashMap();
     Integer ONE = new Integer(1);
     for (int i=0, n=args.length; i<n; i++) {
       String key = args[i];
       Integer frequency = (Integer)map.get(key);
       if (frequency == null) {
         frequency = ONE;
       } else {
         int value = frequency.intValue();
         frequency = new Integer(value + 1);
       }
       map.put(key, frequency);
     }
     System.out.println(map);
     Map sortedMap = new TreeMap(map);
     System.out.println(sortedMap);
   }
 }
```

Set Implementations

- HashSet
  - a Set backed by a hash table
- TreeSet
  - A balanced binary tree implementation
  - Imposes an ordering on its elements

List Implementations

- ArrayList
  - a resizable-array implementation like Vector
    - unsynchronized, and without legacy methods

- LinkedList
  - a doubly-linked list implementation
  - May provide better performance than ArrayList
    - if elements frequently inserted/deleted within the List
  - For queues and double-ended queues (deques)

- Vector
  - a synchronized resizable-array implementation of a List with additional "legacy" methods.

Map Implementations

- HashMap
  - A hash table implementation of Map
  - Like Hashtable, but supports null keys & values
- TreeMap
  - A balanced binary tree implementation
  - Imposes an ordering on its elements
- Hashtable
  - Synchronized hash table implementation of Map interface, with additional "legacy" methods.

WeakHashMap

- WeakHashMap
  - Special-purpose implementation of Map interface storing only weak references to its keys
  - Allows key-value pairs to be garbage-collected when the key is no longer referenced outside of the WeakHashMap
  - Useful for implementing "registry-like" data structures, where the utility of an entry vanishes when its key is no longer reachable by any thread.

Sorting using JCF

- Collections.sort() static method // Collections class
- SortedSet, SortedMap interfaces
  - Collections that keep their elements sorted
  - Iterators are guaranteed to traverse in sorted order
- Ordered Collection Implementations
  - TreeSet, TreeMap

Sorting using JCF

- Comparable interface
  - Must be implemented by all elements in SortedSet
  - Must be implemented by all keys in SortedMap
  - Method: **`int compareTo(Object o)`**
  - Defines "natural order" for that object class
- Comparator interface
  - Defines a function that compares two objects
  - Can design custom ordering scheme
  - Method: **`int compare(Object o1, Object o2)`**

Unsupported Operations

- An implementation class may elect not to support a particular method of the interface
- UnsupportedOperationException is a runtime (unchecked) exception

Utility Classes

- Collections class
- Static methods:
  - sort(List)
  - binarySearch(List, Object)
  - reverse(List)
  - shuffle(List)
  - fill(List, Object)
  - copy(List dest, List src)
  - min(Collection)
  - max(Collection)
  - synchronizedX, unmodifiableX factory methods